MDM: The GPU Memory Divergence Mode

Computer Architecture
– Research Paper Presentation Assignment

Changhee Hong
Student ID: 202204495

Contents

1. Core Concepts and Fundamental Ideas

What is Analytical Modeling?

- It is a technique that captures the core performance phenomena of an architecture through a set of mathematical equations.
- It enables early-stage design space exploration several orders of magnitude faster than cycle-accurate simulation by identifying key performance trends.

The Importance of Analytical Modeling

- The speed advantage is void if the conclusions are misleading due to model inaccuracies, so it is crucial to accurately capture key performance trends across a broad range of applications and architectural configurations.
- Analytical models are derived from a fundamental understanding of the architecture, thus providing deep insight like a "white box" and have a low initial cost when exploring large design spaces.

1. Core Concepts and Fundamental Ideas

What are Memory Divergent (MD) Applications?

- They are applications where the memory access patterns of concurrently executing threads are divergent, accessing different cache lines.
- This is mainly caused by strided or data-dependent access patterns and is commonly found in modern workloads like machine learning and data analytics.
- The paper defines an application as MD if it has more than 10 Divergent Loads Per Kilo Instructions (DPKI > 10).

Difference from Non-Memory Divergent (NMD) Applications

- **NMD (e.g., Grid Stride = 1):** Threads within a warp access the same cache line, and their accesses are coalesced. It has high spatial locality.
- **MD (e.g., Grid Stride = 32):** Threads within a warp access different cache lines, causing cache requests to be divergent. It has very poor spatial locality.

Listing 1: Example kernel with strided access pattern.

```
int ix = blockIdx.x*blockDim.x+threadIdx.x;
__shared__ output[blockDim.x][N];
for (i=0; i<N; i++){
  int index = GS*ix+i;       // Compute index
  float t   = input[index]; // Load value
  output[ix][i] = t*t;
}
```

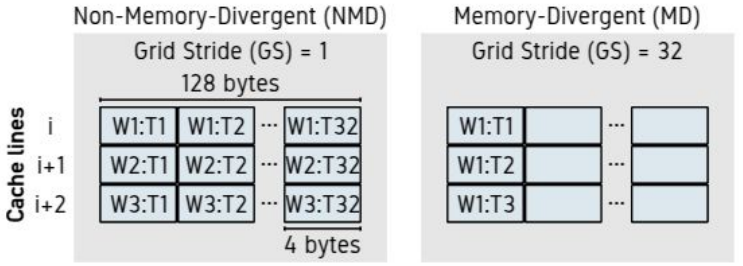

Fig. 2: Cache behavior for the example kernel in Listing 1.

Hankuk University of Foreign Studies

1. Core Concepts and Fundamental Ideas

The Role and Limits of TLP and MSHRs

- **Thread-Level Parallelism (TLP)**
  - To hide memory access latency, a GPU utilizes TLP by executing other independent warps when one warp is stalled due to memory access.
- **MSHR (Miss Status Holding Registers)**
  - These are hardware registers that store information about an L1 cache miss when it occurs.
  - A cache can handle as many concurrent cache misses as it has MSHRs.
- **TLP Breakdown in MD Applications**
  - Due to the poor spatial locality of MD applications, the number of concurrent cache misses exceeds the number of MSHRs.
  - This causes the L1 cache to block, unable to process further misses, which cripples the GPU's ability to hide latency through TLP.

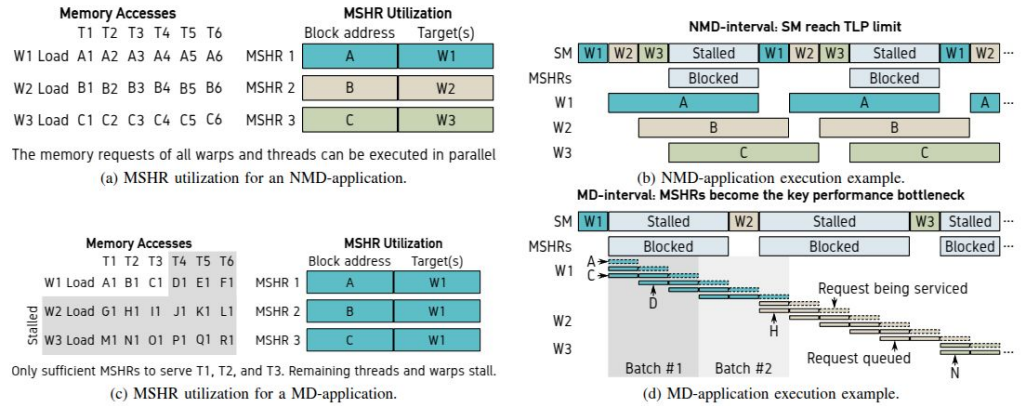

Fig. 4: Example explaining why MSHR utilization results in significantly different performance-related behavior for NMD and MD-applications. *MD-applications puts immense pressure on the L1 cache MSHRs and thereby severely limit the GPU's ability to use TLP to hide memory latencies.*

2. Problem Statement and Motivation

Limitations of Existing Analytical Models

- **Core Problem:** The state-of-the-art analytical GPU performance model, GPUMech, is highly inaccurate for predicting the performance of memory-divergent (MD) applications.
- **Severity of Error:**
  - GPUMech shows an average performance prediction error of 298% for MD applications compared to simulation.
  - While accurate for NMD applications, it shows significant inaccuracy for MD applications.
- **Practicality Issue:** GPUMech relies on slow functional simulation to collect model inputs, which reduces its practicality.

2. Problem Statement and Motivation

Importance and Motivation of the Problem

- **Prevalence of MD Applications:**
  - MD applications are very important and prevalent in modern GPU workloads such as machine learning and data analytics.
  - 38% (21 out of 56) of the applications in major benchmark suites analyzed in the paper were classified as MD.
- **The Need for an Accurate Model:**
  - Inaccurate models can lead to misleading conclusions during design space exploration, causing architects to make poor design decisions.
  - Therefore, an analytical model that accurately captures the key performance behavior of MD applications is urgently needed.

TABLE I: MD-applications across widely used GPU benchmark suites. *MD-applications are common.*

Suite	Ref.	#MD-app.	#NMD-app.	Sum
Rodinia	[5]	4	12	16
Tango	[20]	2	6	8
LonestarGPU	[3]	4	2	6
Polybench	[11]	4	8	12
Mars	[13]	6	0	6
Parboil	[32]	1	7	8
Sum		**21**	**35**	**56**

2. Problem Statement and Motivation

The Root Cause of Inaccuracy

- Previous work (including GPUMech) failed to properly model two key performance degradation factors in MD applications.
  1. **L1 Cache Blocking due to Lack of MSHRs:** They do not model the phenomenon where concurrent cache misses exceed the number of MSHRs, causing the cache to block and TLP to break down.
  2. **NoC/DRAM Queuing Delays:** They do not accurately account for the severe congestion and resulting queuing delays in the Network-on-Chip (NoC) and DRAM subsystems caused by the flood of memory requests from MD applications.

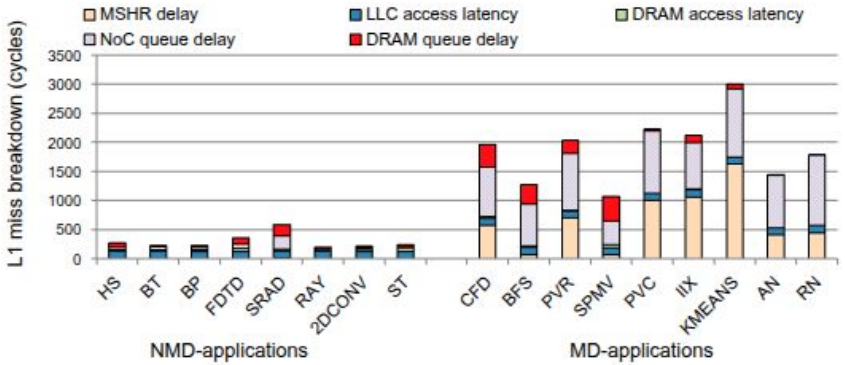

Fig. 3: L1 miss latency breakdown for selected GPU compute applications. *Delays due to insufficient MSHRs as well as queuing delays in the NoC and DRAM subsystem significantly affect the overall memory access latency of MD-applications, while NMD-applications are hardly affected.*

3. Main Contributions of the Paper

MDM's 3 Key Contributions

- **1. Identified Key Performance Mechanisms:** The paper identified the main architectural mechanisms that determine the performance of memory-divergent applications.
    - It identified that poor spatial locality leads to a lack of MSHRs, which results in cache misses being processed in batches and prevents the SM from using TLP to hide memory latency.
- **2. Proposed a New Analytical Model, MDM:** It proposed the Memory Divergence Model (MDM), which faithfully models this batching behavior and NoC/DRAM queuing delays.
- **3. Achieved Breakthrough Improvements in Accuracy, Practicality, and Scope:** It significantly improved the model's accuracy, practicality, and scope compared to the state-of-the-art.

3. Main Contributions of the Paper

Specific Improvements

- **Accuracy:**
  - Compared to detailed simulation, MDM showed an average prediction error of 13.9%, whereas GPUMech had an error of 162%.
  - Compared to real hardware, MDM had a 40% prediction error, while GPUMech had a 164% error.
- **Practicality:**
  - It improved model evaluation speed by 6.1x by collecting model inputs through binary instrumentation instead of functional simulation.
- **Scope:**
  - It gained the ability to model not only traditional NMD applications but also the prevalent and hard-to-predict MD applications.

4. System Design and Methodology

MDM-based Performance Prediction Flow

- MDM is based on interval modeling. The entire prediction process consists of 4 steps, as shown in Figure 1.
  1. **Trace Collection (one-time cost):** Collect instruction traces via binary instrumentation on real hardware or functional simulation.
  2. **Representative Warp Selection (one-time cost):** Based on the observation that the execution behaviors of warps are similar, a representative warp is selected using architecture-independent characteristics like instruction mix.
  3. **Cache Simulation (recurring cost):** The load/store traces of all warps are fed into a cache simulator to obtain miss rates.
  4. **MDM Model Application (recurring cost):** The interval information and miss rates are provided to the MDM model to predict the final performance (IPC).

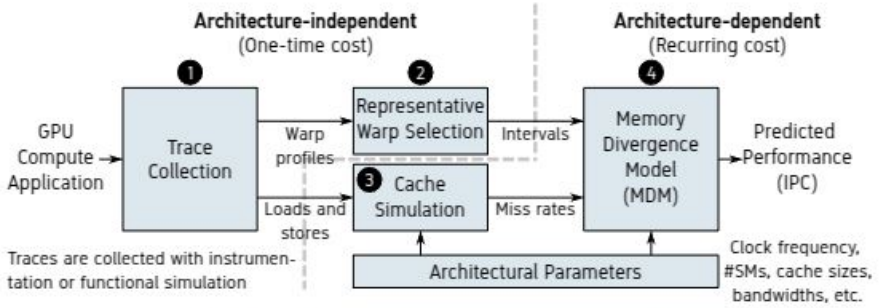

Fig. 1: MDM-based performance prediction.

collecting model inputs through binary instrumentation as opposed to functional simulation.

4. System Design and Methodology

MDM's Core Prediction Equation

- MDM predicts the execution cycles of an interval *i* (Si) by adding the stall cycles due to MSHR, NoC, and DRAM to the base execution cycles (Ci).
- The IPC per Streaming Multiprocessor (IPCSM) is calculated by the following Equation (1), where *W* is the number of concurrently executed warps on an SM.

$$\mathrm{IPC}^{\mathrm{SM}} = W \times \frac{\sum_{i=0}^{\#\mathrm{Intervals}} \#\mathrm{Instructions}_i}{\sum_{i=0}^{\#\mathrm{Intervals}} C_i + S_i^{\mathrm{MSHR}} + S_i^{\mathrm{NoC}} + S_i^{\mathrm{DRAM}}}. \qquad (1)$$

4. System Design and Methodology

MD/NMD Interval Classification

- MDM first classifies each interval as either MD or NMD. This is to determine if the interval has the potential to make MSHRs a performance bottleneck.
- An interval is considered an MD-interval if the number of concurrent read misses exceeds the number of L1 cache MSHRs (Equation 2).

$$M^{\text{Read}} \times W > \#\text{MSHRs}. \qquad (2)$$

  - MRead: Number of read misses in the representative warp
  - W: Number of concurrently executing warps
  - #MSHRs: Number of L1 cache MSHRs

4. System Design and Methodology

Modeling MSHR Blocking (The Batching Model)

- **Observation:** In MD-intervals, due to a lack of MSHRs, memory requests are processed in batches, with each batch size being equal to the number of MSHRs.
- **Modeling:** The stall cycles due to MSHR contention (SMSHR) are calculated by Equation (6).

$$S^{\text{MSHR}} = \begin{cases} (\lceil \frac{M^{\text{Read}} \times W}{\#\text{MSHRs}} \rceil - 1) \times S^{\text{Mem}}, & \text{if MD-interval} \\ 0, & \text{otherwise.} \end{cases} \quad (6)$$

  - The number of required batches is calculated by dividing the total number of read misses by the number of MSHRs.
  - We subtract one because the latency of the final batch is covered by the queuing model.
  - SMem is the L1 miss stall cycle considering both NoC and DRAM contention.

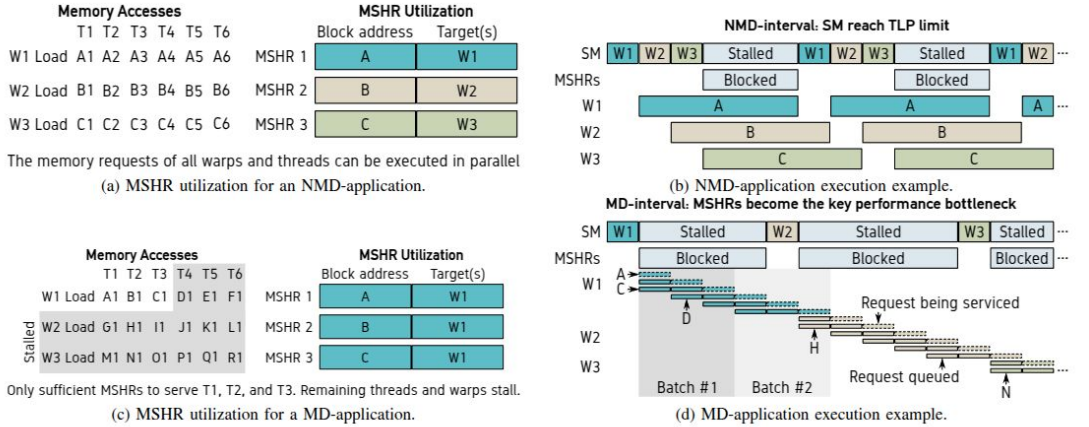

Fig. 4: Example explaining why MSHR utilization results in significantly different performance-related behavior for NMD and MD-applications. *MD-applications puts immense pressure on the L1 cache MSHRs and thereby severely limit the GPU's ability to use TLP to hide memory latencies.*

4. System Design and Methodology

Modeling Memory Contention (The Queuing Model)

- **Observation:**
  - In NMD-intervals, the SM can hide part of the queuing delay using TLP. MDM uses a heuristic that assumes a stall for half of this delay.
  - In MD-intervals, TLP breaks down, and the application is exposed to the full queuing latency. This happens when the NoC becomes saturated.
- **Modeling:** The stall cycles due to NoC (SNoC) are calculated as in Equation (9). DRAM stalls are calculated similarly.

$$S^{\mathrm{NoC}} = \begin{cases} \#\mathrm{SMs} \times M \times L^{\mathrm{NoCService}}, & \text{if MD and NoC saturated} \\ 0.5 \times \#\mathrm{SMs} \times M \times L^{\mathrm{NoCService}}, & \text{otherwise.} \end{cases} \tag{9}$$

  - M: Average number of concurrent L1 misses (calculated by Equation 3).
    - MRite: Number of write misses in the representative warp

$$M = \min(M^{\mathrm{Read}} \times W, \#\mathrm{MSHRs}) + M^{\mathrm{Write}} \times W. \tag{3}$$

  - LNoCService: NoC service latency (calculated by Equation 7).
    - This value is determined by the clock frequency (f), cache block size (BlockSize), and NoC bandwidth (BNoC). DRAM service latency is calculated similarly

$$L^{\mathrm{NoCService}} = f \times \frac{\mathrm{BlockSize}}{B^{\mathrm{NoC}}}. \tag{7}$$

4. System Design and Methodology

Experimental Tools and Environment

- **Simulation:**
  - Used the GPGPU-sim 3.2 cycle-accurate simulator.
  - Based on a GPU configuration similar to Nvidia's Pascal architecture, including 28 SMs and 1050 GB/s NoC bandwidth.
- **Real Hardware:**
  - Collected traces on an NVIDIA GeForce GTX 1080 GPU.
  - Extended the NVBit binary instrumentation framework to capture per-warp memory address and dynamic instruction traces.
- **Benchmarks:**
  - Selected 8 NMD and 9 MD applications from major benchmark suites including Rodinia, Parboil, MARS, and Tango.

5. Results and Evaluation

Model Accuracy on Baseline Configuration (vs. Simulation)

- **Key Result (Figure 5):** MDM shows vastly superior accuracy compared to GPUMech across all benchmarks.
  - **Overall Average Error:** MDM 13.9% vs. GPUMech 162%.
  - **MD Application Average Error:** MDM 18% vs. GPUMech 298%. MDM is 16.5x more accurate.
  - **NMD Application Average Error:** Both MDM and GPUMech show similar accuracy at around 9%.

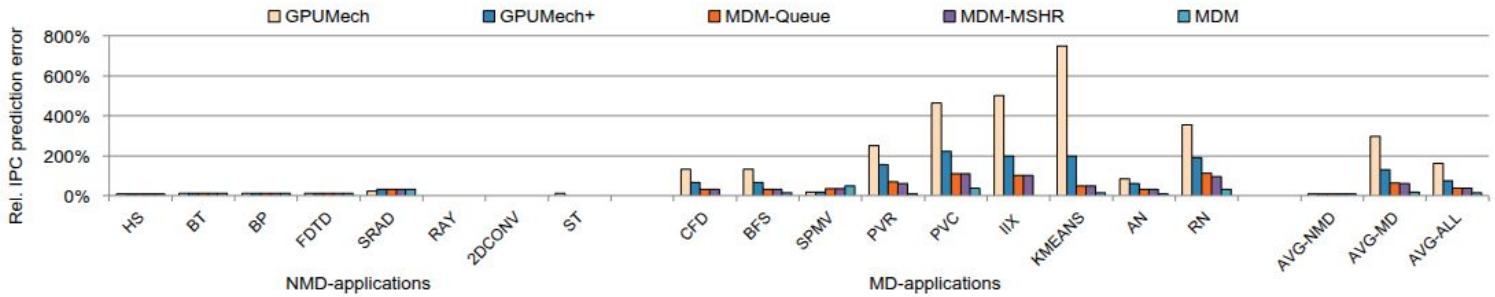

Fig. 5: IPC prediction error for our NMD and MD-benchmarks under different performance models. *MDM significantly reduces the prediction error for the MD-applications.*

5. Results and Evaluation

Importance of Model Components

- **Analysis of MDM-Queue and MDM-MSHR (Figure 5):**
  - Applying only MDM's queuing model (MDM-Queue) or MSHR batching model (MDM-MSHR) improved performance over GPUMech+ but still resulted in high errors of 63% and 60.3%, respectively.
  - This indicates that modeling **both** queuing effects and MSHR behavior is critical to achieve high accuracy.

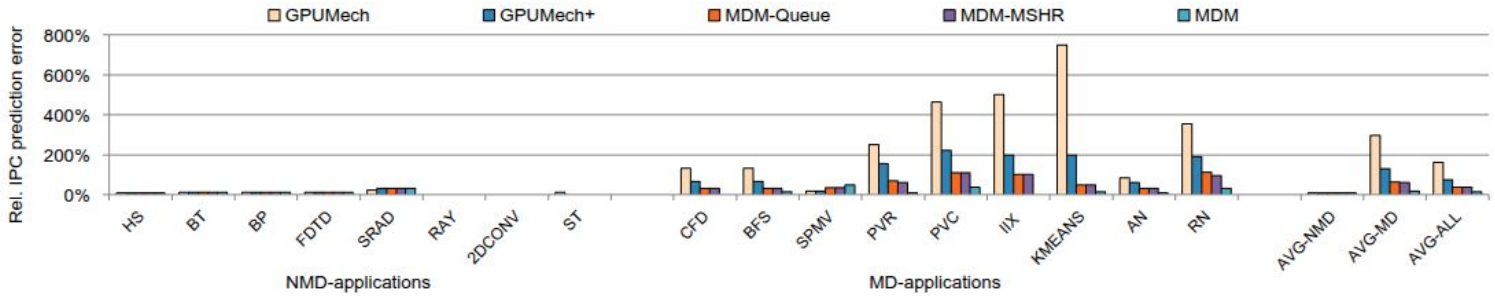

Fig. 5: IPC prediction error for our NMD and MD-benchmarks under different performance models. *MDM significantly reduces the prediction error for the MD-applications.*

5. Results and Evaluation

Sensitivity Analysis: NoC/DRAM Bandwidth

- **NoC Bandwidth (Figure 6):**
  - GPUMech does not model NoC bandwidth, leading to very large errors at low bandwidths.
  - MDM considers the pressure MSHRs put on NoC bandwidth, maintaining high accuracy across the entire range of NoC bandwidths.
- **DRAM Bandwidth (Figure 7):**
  - GPUMech's error increases with DRAM bandwidth because it doesn't model the resulting pressure on the NoC.
  - MDM counters this trend and maintains high accuracy by synergistically modeling saturation and batching behavior.

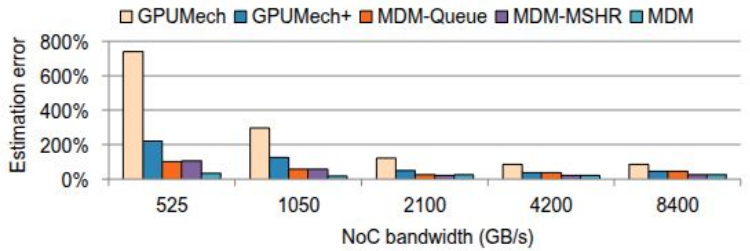

Fig. 6: Prediction error as a function of NoC bandwidth for the MD-applications.

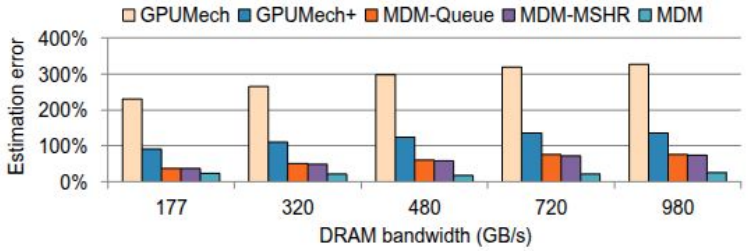

Fig. 7: Prediction error as a function of DRAM bandwidth for the MD-applications.

Hankuk University of Foreign Studies

5. Results and Evaluation

Sensitivity Analysis: MSHR Count and SM Count

- **MSHR Count (Figure 8):**
  - GPUMech's error worsens as MSHR count increases, because more MSHRs lead to more concurrent requests and thus higher NoC/DRAM queuing delays.
  - MDM achieves the highest accuracy across the full range of MSHR counts.
- **SM Count (Figure 9):**
  - Increasing the number of SMs increases NoC and DRAM contention, causing GPUMech's error to soar from 124% to 450%.
  - MDM combines its model enhancements to reduce the error to less than 26% on average across different SM counts.

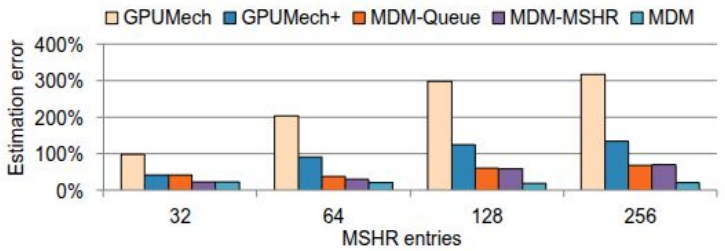

Fig. 8: Prediction error as a function of the number of MSHR entries for the MD-applications.

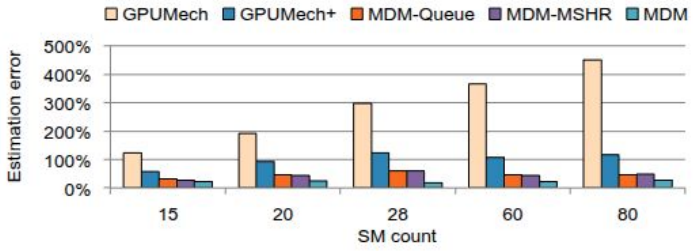

Fig. 9: Prediction error as a function of SM count for the MD-applications.

5. Results and Evaluation

Real Hardware Validation (vs. GTX 1080)

- **Superiority of Binary Instrumentation (Figure 10):**
  - For GPUMech, using binary instrumentation instead of functional simulation reduced the average error from 260% to 164%. This is because profiling at the native instruction level is more accurate.
- **MDM's Hardware Accuracy:**
  - MDM demonstrated high accuracy against real hardware with an average prediction error of 40%, proving its accuracy not just in simulation but also in a real-world environment

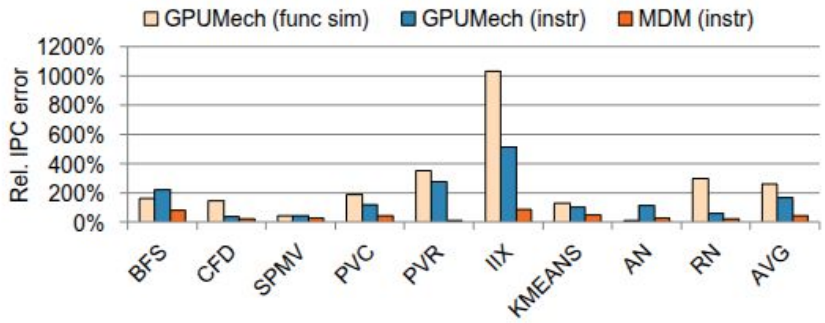

Fig. 10: Hardware validation: relative IPC prediction error for GPUMech and MDM compared to real hardware. *MDM achieves high prediction accuracy compared to real hardware with an average prediction error of 40% compared to 164% for GPUMech (using binary instrumentation).*

5. Results and Evaluation

Case Study 1: Design Space Exploration

- **NMD (BT) vs. MD (CFD) Applications (Figures 11, 12):**
  - For the NMD app, both GPUMech and MDM predicted the performance trend well.
  - For the MD app (CFD), GPUMech grossly overestimated the performance improvement from increasing SM count and DRAM bandwidth, leading to a misleading conclusion.
  - In contrast, MDM accurately predicted that the performance improvement would be small, in line with simulation results, highlighting the need for an accurate model.

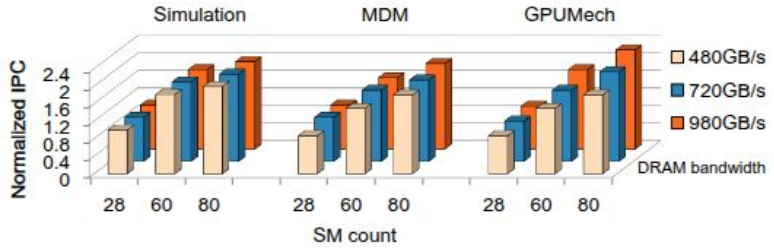

Fig. 11: Normalized performance for BT as a function of SM count and DRAM bandwidth. We normalize to the simulation results at 28 SMs and 480 GB/s DRAM bandwidth. *Both GPUMech and MDM capture the performance trend.*

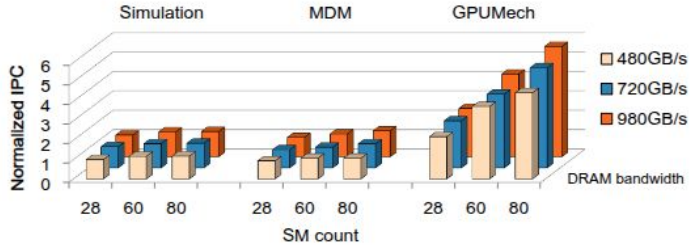

Fig. 12: Normalized performance for CFD as a function of SM count and DRAM bandwidth. All results are normalized to the simulation results at 28 SMs and 480 GB/s DRAM bandwidth. *GPUMech not only leads to high prediction errors, it also over-predicts the performance speedup with more SMs and memory bandwidth, in contrast to MDM.*

5. Results and Evaluation

Case Studies 2 & 3: DVFS and Model Validation

- **DVFS Performance Prediction (Figure 13):**
  - Despite being a general-purpose model, MDM achieved accuracy nearly on par with CRISP, a special-purpose model for DVFS prediction (MDM 4.6% vs CRISP 3.7% error). GPUMech had a 28% error.
- **Model Observation Validation (Figure 14):**
  - MDM's CPI (Cycles Per Instruction) breakdown confirmed that MD application performance is indeed sensitive to MSHR blocking and NoC/DRAM queuing delay.

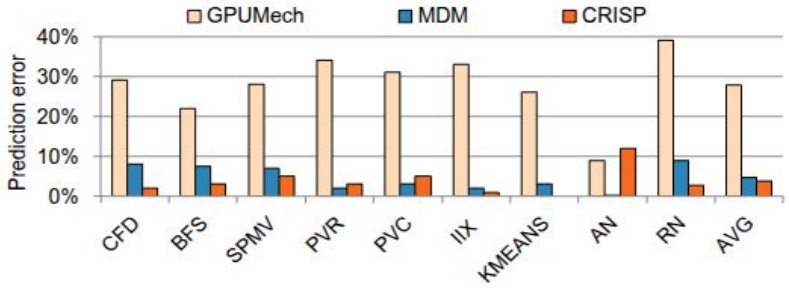

Fig. 13: Error when predicting the relative performance difference at 1.4 GHz versus 2 GHz for the MD-applications for GPUMech, CRISP and MDM. *The general-purpose MDM model achieves similar accuracy as the special-purpose CRISP.*

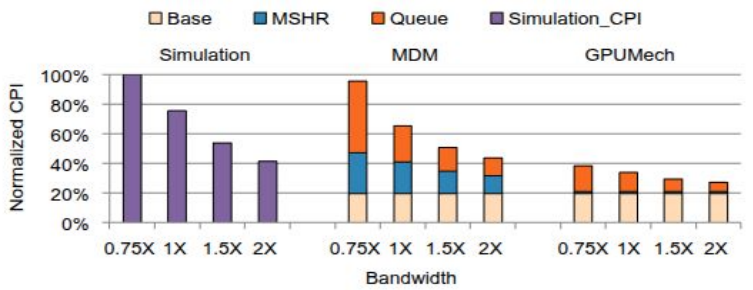

Fig. 14: Normalized CPI as a function of NoC/DRAM bandwidth for the memory-divergent BFS benchmark. *MDM accurately captures MSHR batching and NoC/DRAM queueing delays, in contrast to GPUMech.*

Hankuk University of Foreign Studies

5. Results and Evaluation

Case Study 4: Modeling a Streaming L1 Cache

- **Streaming Cache in Volta Architecture:**
  - This cache has a very large number of MSHRs (e.g., 4096), which practically eliminates L1 blocking.
  - However, batching behavior still occurs because the queues in the NoC are finite.
- **Result (Figure 15):**
  - MDM models the batching behavior caused by NoC saturation, proving it works well even in a streaming L1 cache environment and improving accuracy by 8.66x compared to GPUMech.

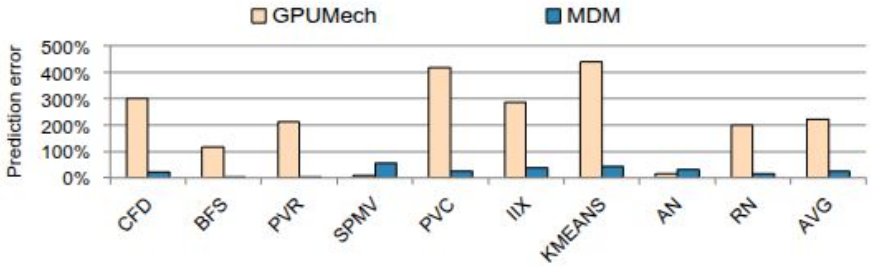

Fig. 15: Relative IPC prediction error with a streaming L1 cache. *MDM improves accuracy compared to GPUMech because it models batching behavior caused by NoC saturation.*

6. Strengths and Weaknesses

Strengths of MDM (Pros)

- **High Accuracy and Broad Scope:** Drastically improves accuracy for hard-to-predict MD applications while maintaining the accuracy of existing models for NMD applications.
- **Excellent Practicality:** It is 6.1x faster than functional simulation-based models by using binary instrumentation, enabling large design space explorations that could take months or years to be done in days.
- **Provides Deep Insight:** Unlike "black box" machine learning models, MDM is a "white box" analytical model that explains the causes of performance bottlenecks, such as MSHR batching and NoC saturation.
- **Thorough Validation:** It has been extensively validated against detailed simulation, real hardware, and across a wide range of architectural parameters (NoC/DRAM bandwidth, MSHR/SM count).

6. Strengths and Weaknesses

Weaknesses and Limitations of MDM (Cons)

- **Remaining Prediction Error:** While much improved, an average error of 13.9% against simulation and 40% against real hardware still exists. The paper does not deeply analyze the specific sources of this remaining error.
- **Use of Heuristics:** The model uses a "half-latency" heuristic for modeling queuing delay in NMD-intervals. While described as a reasonable assumption, it is a simplification rather than a first-principles model.
- **Dependency on Trace Collection:** Before applying the model, a one-time trace collection process is required for each application, which can still be a time-consuming task.

7. Future Work and Research Directions

Proposals for Future Research

- **Model Refinement and Error Analysis:**
  - Research is needed to identify the sources of the remaining 14-40% prediction error and improve the model. For example, the overlap between TLP and memory access could be modeled more precisely, or the 'half-latency' heuristic could be replaced with a more advanced model.
- **Expanding Model Scope:**
  - Currently, MDM focuses on memory divergence. It could be extended into a unified model that also covers other types of GPU bottlenecks, such as shared memory bank conflicts or instruction pipeline stalls.
- **Exploring Real-time Applicability:**
  - MDM was designed for offline design space exploration. Research into simplifying the model for use in runtime optimizations (e.g., dynamic cache bypassing, thread block scheduling), similar to CRISP, would be promising.
- **Combining Analytical Models with Machine Learning:**
  - Instead of a threshold-based approach (MRead×W>#MSHRs) for classifying MD/NMD intervals, machine learning could be used to perform more nuanced interval classification or even predict stall parameters directly, combining the insight of analytical models with the predictive power of ML.

Thank you :)